Date __/__/__
Intimate

//Code - (Sliding Window) -

```
public int maxSubarrayLength(int nums[], int k){
    int ans = 0;
    HashMap<Integer, Integer> mp =
                        new HashMap<>();
    int left = 0;
    int right = 0;
    for(int
    int n = nums.length;

    while (~~j<n~~) (right < n){
        mp.put(nums[right],
            mp.getOrDefault(nums[right], 0)+1);

        while (left < right &&
            mp.getOrDefault(nums[right],0) > k){

            mp.put(nums[left], map.get(nums[right])-1);
```

Code (Sliding Window) –

```
public int maxSubarrayLength(int nums[], int k) {
    int n = nums.length;

    Map<Integer, Integer> map = new HashMap<>();

    int i = 0;    int j = 0;
    int result = 0;

    while (j<n){
        map.put(nums[j], map.getOrDefault(nums[j],0)+1);

        while (i<j && map.getOrDefault(nums[j], 0) >k) {
            map.put(nums[i], map.get(nums[i]) - 1);
            if (map.get(nums[i]) == 0) {
                map.remove(nums[i]);
            }
            i++;
        }
        result = Math.max(result, j-i+1);
        j++;
    }
    return result;
}
```

TC- O(2N)
SC- O(N)

Date / /
Intimate

18. Count Subarrays Where Max Element Appears at least K Times -

nums = {1, 3, 2, 3, 3}, K = 2

Output = 6 {1, 3, 2, 3}, {1, 3, 2, 3, 3}
{3, 2, 3}, {3, 2, 3, 3}
{2, 3, 3}, {3, 3} [maxElement = 3]

DRY. RUN#

nums = | 1 | 3 | 2 | 3 | 3 | , K = 2

maxElement = 3.

Date __/__/__
Intimate

// Code (Sliding Window) -

```
public long countSubarrays(int[] nums, int k){
    int maxE = Integer.MIN_VALUE;

    for(int i=0; i<nums.length; i++){
        if(nums[i] > maxE){
            maxE = nums[i];
        }
    }
    int n = nums.length;  int i = 0;  int j = 0;
    long result = 0;
    int countMax = 0;

    while(j<n){
        if( nums[j] == maxE ){
            countMax++;
        }
        while( countMax >= k){
            result += n-j;

            if(nums[i] == maxE ){
                countMax--;
            }
            i++;
        }
        j++;
    }
    return result;
}
```

Date __/__/__
Intimate

19. Subarrays with K Different Integers -

	0	1	2	3	4
nums =	1	2	1	2	3

Output - 7 {1,2}, {2,1} {1,2} {2,3}
{1,2,1} {2,1,2} {1,2,1,2}

DRY RUN

nums =	1	2	1	2	3	, K = 2

~~two~~ बार K के लिए -

count = 1+2+3+4+2 = 12

K=2, K=1

{1}, {1,2} {2}
{1,2,1} {2,1} {1}
{2,1,2}, {1,2} {2}

map

1	1
2	1
3	1

एक बार K-1 के लिए -

K-1 = 1

* return Sliding_window(K) - Sliding_window(K-1)

Date __/__/__
Intimate

// Code - (It has one pass solution too).

```
public int slidingWindow(int[] nums, int k){
    HashMap<Integer, Integer> mp = new HashMap<>();

    int n = nums.length;
    int i = 0;
    int j = 0;
    int count = 0;

    while(j < n){
        mp.put(nums[j], mp.getOrDefault(nums[j],0)+1);

        while(mp.size() > k){
            mp.put(nums[i], mp.get(nums[i])-1);

            if(mp.get(nums[i])==0){
                mp.remove(nums[i]);
            }

            i++;
        }

        count += (j-i+1);
        j++;
    }

    return count;
}
public int subarraysWithKDistinct(int nums[], int k){
    return slidingWindow(nums, k) -
                slidingWindow(nums, k-1);
}
```

Similar :- Minimum Size Subarray Sum

Date / /
Intimate

20. Get Equal Substrings Within Budget -

(1) Example - s = "a b c d" maxCost = 3

t = "b c d f"

Output - 3

(2) Example - s = "a b c d" maxCost = 3

t = "c d e f"

Output - 1

Dry Run

s = "a b c d" maxCost = 3

t = "b c d f"

i, j

s[i:j] = j - i + 1 = 3 - 2 + 1 ⇒ 2

length = 0 V/ 3

Code:- (Sliding Window) —

```
public int equalSubstring(String s, String t, int maxCost) {
    int n = s.length();

    int maxLen = 0;
    int currCost = 0;

    int i = 0, j = 0;
    while (j < n) {
        currCost += Math.abs(s.charAt(j) - t.charAt(j));

        while (currCost > maxCost) {
            currCost -= Math.abs(s.charAt(i) - t.charAt(i));
            i++;
        }

        maxLen = Math.max(maxLen, j-i+1);
        j++;
    }
    return maxLen;
}
```

Date / /
Intimate

21. Find Subarray With Bitwise AND Closest to K -

Ex. - nums = {1, 2, 4, 5} K = 3

Output = 1

{4, 5}

abs (4 - 3) => (1) ↳ smallest possible

4 => 100
5 => 101
& 100 = 4

Brute Force

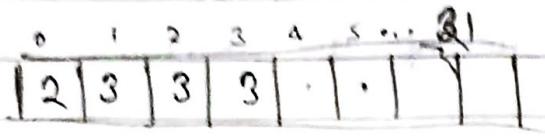
(•) Find all Subarrays O(N^3)
Find AND
Find min {abs (K - AND)}

```
for (i = 0; i<n; i++) {
    for (j = 0; j<n; j++) {
        AND = [i to j] or AND
        result = min (result, abs (K - AND));
    }
}
```

```
for (i = 0; i<n; i++) {
    AND = nums[i];                O(N^2)
    for (j = 0; j<n; j++) {
        AND = (AND & nums[j]);
        result = min (result, abs (K - AND));
    }
}
```

Date / /
Intimate

DRY RUN



// Code (Sliding Window + Bit)

```
private void updateFreq(int op, int val, int freqBits[]) {
    int i = 0;
    while (val > 0) {
        if ((val & 1) == 1) {
            freqBits[i] += op;
        }
        val /= 2;
        i++;
    }
}

public int minimumDifference(int nums[], int k) {
    int[] freqBits = new int[32];
    Arrays.fill(freqBits, 0);

    int n = nums.length;
    int i = 0;
    int j = 0;
    int windowAnd = nums[0];
    int result = Integer.MAX_VALUE;

    while (j < n) {
        windowAnd = windowAnd & nums[j];
        updateFreq(1, nums[j], freqBits);

        result = Math.min(result, Math.abs(k - windowAnd));
        if (windowAnd > k) {
            j++;
        } else if (windowAnd == k) {
            return 0;
        }
```

Date / /
Intimate

```
else {
    while (i <= j && windowAnd < k) {
        updateFreq(-1, nums[i], freqBits);
        i++;
        windowAnd = 0;
        for (int b = 0; b < 32; b++) {
            if ((j - i + 1) == freqBits[b]) {
                windowAnd = windowAnd | (1 << b);
            }
        }

        result = Math.min(result, Math.abs(k - windowAnd));
    }
    j++;
}
}
return result;
}
```

22. Grumpy Bookstore Owner –

Example – customers = {1, 0, 1, 2, 1, 1, 7, 5}

grumpy = {0, 1, 0, 1, 0, 1, 0, 1}

minutes = 3 (Fixed size + consecutive) ≈ Sliding window

Output – 16

Dry Run

Code Intimate

// Code :-

```
public int maxSatisfied(int[] customers, int[] grumpy,
                        int minutes) {
    int n = customers.length;
    int i = 0;
    int j = 0;
    int currentGain = 0;
    int maxGain = 0;

    while (j < minutes) {
        if (grumpy[j] == 1) {
            currentGain += customers[j];
        }
        j++;
    }

    maxGain = currentGain;
    while (j < n) {
        if (grumpy[j] == 1) {
            currentGain += customers[j];
        }
        if (grumpy[i] == 1) {
            currentGain -= customers[i];
        }
        maxGain = Math.max(maxGain, currentGain);
        i++;
        j++;
    }
    int alwaysSatisfied = 0;
    for (int k = 0; k < n; k++) {
        if (grumpy[k] == 0) {
            alwaysSatisfied += customers[k];
        }
    }
    return alwaysSatisfied + maxGain;
```

"GEE"

Q1. Longest Increasing Subsequence with Absolute difference less than or equal to limit -

Example - { nums = {3, 2, 4, 7, 2} limit = 4
Output = 3

Example - { nums = {10, 1, 2, 4, 7, 2} limit = 5
Output = 4

Explanation :-

{10, 1, 2, 4, 7, 2} Diff limit = 11

Subsequence with max element } Diff ⇒ 10 - 1 ⇒ 9)
Subsequence with min element } Diff ⇒ 7 - 2 ⇒ 5) Not valid
Subsequence is valid.

- Q2. Same as -

nums = {2, 1, 4, 4, 3, 10} , limit = 6

- minHeap -

(1, 1)
(2, 0)
3
4
2
10

Q, min heap → {element, idx}
Q, max heap → {element, idx}
Q, i, j → sliding window

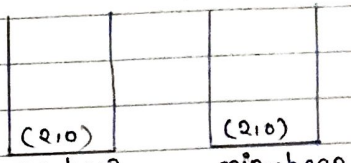
Date __/__/__
Intimate

Dry Run –

(1) {2, 1, 4, 4, 3, 10}, limit = 6
i=0 j=1 2 3 4 5

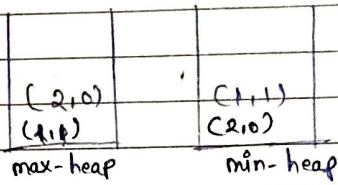
maxlength = ~~0~~ 1

2-2 ⇒ 0 <= limit



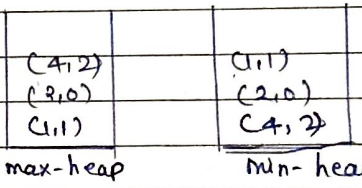
(2) {2, 1, 4, 4, 3, 10}, limit = 6
i=0 j=1
maxlength = 2

2-1 ⇒ 1 <= limit



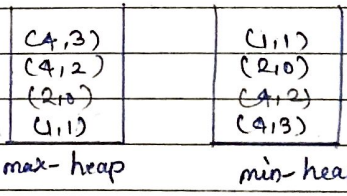
(3) {2, 1, 4, 4, 3, 10}, limit = 6
i=0 j=2
maxlength = 3

4-1 ⇒ 3 <= limit



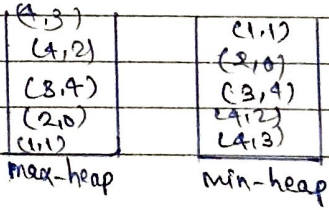
(4) {2, 1, 4, 4, 3, 10}, limit = 6
i=0 j=3
maxlength = 4

4-1 ⇒ 3 <= limit



(5) {2, 1, 4, 4, 3, 10}, limit = 6
i=0 j=4
maxlength = ~~5~~ 4

4-1 = 3 <= limit



Date / /
Intimate

(6) {3, 1, 4, 4, 3, 10}, limit = 6
i j

maxlength = 5 10-1 ⇒ 9 > limit

max-heap	min-heap
(10,5)	~~(1,1)~~
(4,3)	~~(2,0)~~
(4,2)	(3,4)
(3,4)	(4,2)
(2,0)	~~(4,3)~~
(1,1)	(10,5)

(7) {2, 1, 4, 4, 3, 10}, limit = 6
i j

minlength = 5

10-3 ⇒ 7 > limit

max-heap	min-heap
(10,5)	~~(3,4)~~
(4,3)	~~(4,2)~~
(4,2)	~~(4,3)~~
(3,4)	(10,5)
(2,0)	
(1,1)	

(8) {2, 1, 4, 4, 3, 10}, limit = 6
j i

10-10 ⇒ 0 < = limit

maxlength = 5

max-heap	min-heap
(10,5)	
(1,1)	(10,5)

(9) {2, 1, 4, 4, 3, 10}, limit = 6
i j

Out of Bound.

// Sliding window + Heap

```
public int longestSubarray(int[] nums, int limit) {
    int n = nums.length;
    PriorityQueue<int[]> maxPq =
            new PriorityQueue<>((a,b) -> b[0] - a[0]);
    PriorityQueue<int[]> minPq =
            new PriorityQueue<>((a,b) -> a[0] - b[0]);

    int i = 0;
    int j = 0;
    int maxLength = 0;

    while (j < n) {
        maxPq.offer(new int[] {nums[j], j});
        minPq.offer(new int[] {nums[j], j});

        while (maxPq.peek()[0] - minPq.peek()[0] > limit) {
            i = Math.min(maxPq.peek()[1], minPq.peek()[1]) + 1;

            while (maxPq.peek()[1] < i) {
                maxPq.poll();
            }
            while (minPq.peek()[1] < i) {
                minPq.poll();
            }
        }
        maxLength = Math.max(maxLength, j-i+1);
        j++;
    }
    return maxLength;
}
```

Date __/__/__
Intimat

APPROACH-02

nums = {10, 1, 2, 4, 7, 2} , limit = 5

Is there any other data structure which can give me min & max efficiently ??

(1) Duplicate.
(2) Sorted → max, min
(3) Delete element eff.

Java – TreeMap

// Code – Using Sliding window + TreeMap

```
public int longestSubarray(int nums[], int limit){
    int n = nums.length;
    TreeMap<Integer, Integer> st = new TreeMap<>();
    int i = 0;
    int j = 0;
    int maxLength = 0;
    while (j<n) {
        st.put(nums[j], st.getOrDefault(nums[j],0)+1);
        while (st.lastKey() - st.firstKey() > limit ){
            st.put(nums[i], st.get(nums[i])-1);
            if ( st.get(nums[i]) == 0 ){
                st.remove(nums[i]);
            }
            i++;
        }
        maxLength = Math.max(maxLength, j-i+1);
        j++;
    }
    return maxLength;
}
```

Given Circular Array -

24. Minimum Swaps to Group All 1's Together II -

Ex. (1) nums - ~~{0,0}~~ {0,1,0,1,1,0,0}

Output - 1 => {0,0,1,1,1,0,0}

Ex. (2) nums - {0,1,1,1,0,0,1,1,0}

Output - 2

Thought Process

{1, 1, 0, 1} Total ones = 3.

Trick for any problem involving circular subarray :-

(1) Append original array twice

1,1,0,1,1,1,0,1 => 0 Ans.

(2) (4+1) % 5 = 0 {It is alternative}

Date / /
Intimate

// Code:- (Sliding window + without extra space)

```
public int minSwaps(int nums[]){
    int n = nums.length;
    int countOnes = 0;

    for(int i=0; i<n; i++){
        if(nums[i] == 1){
            countOnes++;
        }
    }

    int i = 0, j = 0, currCount = 0, maxCount = 0;

    while(j < 2*n){
        if(nums[j%n] == 1){
            currCount++;
        }
        if(j-i+1 > countOnes){
            currCount -= nums[i%n];
            i++;
        }
        maxCount = Math.max(maxCount, currCount);
        j++;
    }

    return countOnes - maxCount;
}
```

Date / /
Intimate

25. Count Substrings that Satisfy K-Constraint I

Brute force

S = "0 0 0 1 1 1 1" , K = 2
i j

count(0) <= K
count(1) <= K

count0 = 0
count1 = 0

result = 1+1+1+1+1+1+1+1

// Brute force (Generating All Substrings)

Date __/__/__
Intimate

```
public int countKConstraintStrings(String s, int k){
    int n = s.length();
    int result = 0;

    for(int i=0; i<n; i++){
        int count0 = 0;
        int count1 = 0;

        for(int j= i; j<n; j++){
            if (s.charAt(j) == '0'){
                count0++;
            }
            else{
                count1++;
            }
            if(count0 <= k || count1 <= k){
                result += 1;
            } else{
                break;
            }
        }
    }
    return result;
}
```

Date __/__/__
Intimate

Approach - 02 (Sliding window) -

Dry Run:-

Date / /
Intimate

// Code :- (Using Sliding Window) -

```
public int countKConstraintSubstring (String s, int k) {
    int n = s.length();
    int result = 0;
    int count0 = 0;
    int count1 = 0;
    int i = 0;
    int j = 0;

    while (j < n) {
        if (s.charAt(j) == '0') {
            count0++;
        } else {
            count1++;
        }

        while (count0 > k && count1 > k) {
            if (s.charAt(i) == '0') {
                count0--;
            } else {
                count1--;
            }
            i++;
        }

        result +
        result += (j - i + 1);
        j++;
    }
    return result;
}
```

HARD VERSION —

Count Substrings That Satisfy K-Constraint II -

S = "0001111" , K = 2

Queries = { [0,6] , [2,4] }

Output - { 26, 6 }

Dry Run

0	0	0	1	1	1	1
0	1	2	3	4	5	6

, K = 2

i = 0 (low)

j = 4

count 0 =

count 1 =

rightMost

4	6	6	6	6	6	6
0	1	2	3	4	5	6

low = 0

high = 6

4 - low + 1 ⇒ 4 - 0 + 1 ⇒ 5

validIndex = min (high, rightMost[low]);

Q = 5 * (5+1) / 2 j - i + 1

⇒ 15

Date __/__/__
Intimate

Code:- (Sliding window + cumulative Sum)

```
public long[] countKConstraintSubstrings(String s, int k,
                                         int[][] queries) {
    int n = s.length();
    int[] leftMost = new int[n];
    int[] rightMost = new int[n];

    int i = 0;
    int j = 0;
    Map<Character, Integer> mp = new HashMap<>();

    while (j < n) {
        mp.put(s.charAt(j), mp.getOrDefault(s.charAt(j), 0) + 1);

        while (mp.getOrDefault('0', 0) > k &&
               mp.getOrDefault('1', 0) > k) {
            mp.put(s.charAt(i), mp.get(s.charAt(i)) - 1);
            i++;
        }

        leftMost[j] = i;
        j++;
    }

    mp.clear();
    i = n - 1;
    j = n - 1;
```

Date __/__/__
Intimate

```
while (j >= 0) {
    mp.put(s.charAt(j), mp.getOrDefault(s.charAt(j), 0) + 1);

    while (mp.getOrDefault('0', 0) > k && mp.getOrDefault('1', 0) > k) {

        mp.put(s.charAt(i), mp.get(s.charAt(i)) - 1);
        i--;
    }

    rightMost[j] = i;
    j--;
}

int[] tempValidSubstr = new int[n];
for (j = 0; j < n; j++) {
    tempValidSubstr[j] = j - leftMost[j] + 1;
}

long[] cumSum = new long[n];
cumSum[0] = tempValidSubstr[0];
for (i = 1; i < n; i++) {

    cumSum[i] = cumSum[i-1] + tempValidSubstr[i];
}

long[] result = new long[queries.length];
```

```
for (i = 0; i < queries.length; i++ ) {
    int low = queries[i][0];
    int high = queries[i][1];

    int validRightIdx = Math.min(high, rightMost[low]);

    long length = validRightIdx - low + 1;

    long tempResult = length * (length + 1) / 2;

    if ( validRightIdx < high ) {
        tempResult += cumSum[high] -
                      cumSum[validRightIdx];
    }

    result[i] = tempResult;
}

return result;
}
```

Date / / Intimate

26. Permutation in String – (Amazon & Microsoft)

Example:- s1 = "ab"
s2 = "eidbaooo"

Output – True

Brute Force Approach

Dry Run –

(1) Permutation of s1
(2) check in s2

TC- O(n! * n)
SC- O(n)

//Code - (Brute force) -

Date __/__/__
Intimate

```
class Solution {
    private int n;
    private boolean result;

    private void solve(int idx, String s1, String s2) {
        if (idx == n) {
            if (s2.contains(s1)) {
                result = true;
            }
            return;
        }

        //Generate permutations of s1
        char[] charArray = s1.toCharArray();
        for(int i = idx; i<n; i++) {
            swap(charArray, idx, i);

            String newS1 = new String(charArray);

            solve(idx+1, newS1, s2);

            swap(charArray, idx, i);
            if(result) {
                return;
            }
        }
    }
```

Date __/__/____
Intimate

```
private void swap (char[] array, int i, int j) {
        char temp = array[i];
        array[i] = array[j];
        array[j] = temp;
}

public boolean checkInclusion (String s1, String s2) {
        n = s1.length();

        result = false;

        solve(0, s1, s2);

        return result;
}
}
```

Date / / Intimate

Approach - 02 (Sorting & Comparing)

s1 = "adb" ⟶ abd (sorting)
s2 = "bad" ⟶ abd (sorting)

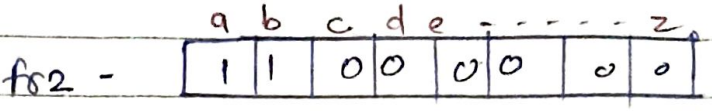
```
public boolean checkInclusion (String s1, String s2){
    int n = s1.length();
    int m = s2.length();

    if (n > m) return false;

    char[] sortedS1 = s1.toCharArray();
    Arrays.sort (sortedS1);

    for (int i=0; i<= m-n; i++){
      String temp = s2.substring (i, i+n);

      char [] sortedTemp = temp.toCharArray();
      Arrays.sort (sortedTemp);

      if (Arrays.equals (sortedTemp, sortedS1)) {
          return true;
      }
    }

    return false;
}
```

Date __/__/__
Intimate

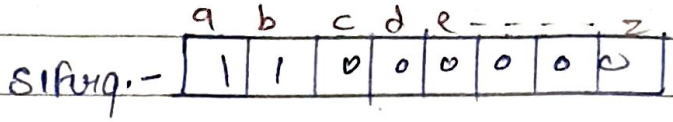
- (Approach-03) - Sliding Window

s1 = "ab"

s2 = "eidbaooo"



DRY RUN! -



// Code (Sliding Window) -

```
public boolean checkInclusion (String s1, String s2) {
    int n = s1.length();
    int m = s2.length();

    if (n > m)  return false;

    int[] s1_freq = new int[26];
    int[] s2_freq = new int[26];

    for (int i = 0; i < n; i++) {
        s1_freq[s1.charAt(i) - 'a']++;
    }

    int i = n, j = 0;
    while (j < m) {
        s2_freq[s2.charAt(j) - 'a']++;

        if (j - i + 1 > n) {
            s2_freq[s2.charAt(i) - 'a']--;
            i++;
        }

        if (Arrays.equals(s1_freq, s2_freq)) {
            return true;
        }

        j++;
    }

    return false;
}
```

Date
Intimate

27. Find the Power of K-Size Subarrays - I -

Example:- nums = [1, 2, 3, 4, 3, 2, 5], K = 3

Output - [3, 4, -1, -1, -1]

{sorted & Consecutive}

Brute force - (1) go to each element & create K size window
(2) check it is sorted & consecutive.

ode (Sliding window) —

```
blic int[] resultArray(int nums[], int k){
    if( nums == null || nums.length < k ){
        return new int[0];
    }

    int arr[] = new int[nums.length - k + 1];
    int i = 0, j = k-1;
    int a = 0;

    while( j < nums.length ){
        if (solved(nums, i, j)){
            arr[a] = nums[j];
        }
        else{
            arr[a] = -1;
        }
        i++;
        j++;
        a++;
    }
    return arr;
}
private boolean solved(int nums[], int start, int end){
    for(int i = start; i<end; i++){
        if( nums[i] + 1 != nums[i+1] ){
            return false;
        }
    }
    return true;
}
```

Date __/__/__
Intimate

~~28. Find the Power of K-size Subarrays I~~

28. Shortest Subarray With Sum at Least K - (+ve & -ve)

Ex:- ① nums = [1], K = 1

Output = 1

② nums = [1, 2], K = 4

Output = -1

③ nums = [2, -1, 2], K = 3

Output :- 3

nums = [84, 37, 32, 40, 95], K = 167

nums = [84, -37, 32, 40, 95], K = 167

Date __/__/__
Intimate

// Code:- (Sliding window + Deque)

```
public int shortestSubarray(int nums[], int k){
    int N = nums.length;

    Deque<Integer> deq = new LinkedList<>();
    long[] cumulativeSum = new long[N];

    int result = Integer.MAX_VALUE;
    int j = 0;

    while(j < N){
        if(j == 0){
            cumulativeSum[j] = nums[j];
        }
        else{
            cumulativeSum[j] = cumulativeSum[j-1] +
                               nums[j];
        }
        if(cumulativeSum[j] >= k){
            result = Math.min(result, j+1);
        }
        while(!deq.isEmpty() &&
cumulativeSum[j] - cumulativeSum[deq.peekFirst()] >= k){
            result = Math.min(result,
                              j - deq.peekFirst());
            deq.pollFirst();
        }
```

Date / /
Intimate

```
while (!deq.isEmpty() &&
    cumulativeSum[j] <= cumulativeSum[deq.peekLast()]) {
    deq.pollLast();
}
deq.offerLast(j);
j++;
}
return result == Integer.MAX_VALUE ? -1 :
    result;
}
```

Date __/__/____
Intimate

29. Maximum Sum of Distinct Subarrays With ~~Legth~~ Length K -

Example:- (1) nums = [1, 5, 4, 2, 9, 9, 9] , K = 3

Output - 15

(1, 5, 4) → 10
(5, 4, 2) → 11
(4, 2, 9) → (15) → Max.
Distinct

(2) nums = [4, 4, 4] , K = 3

Output = 0

Date __/__/__
Intimate

// Code (Sliding + Window) —

```
public long maximumSubarraySum(int[] nums, int k) {
    int i = 0, j = 0;
    long result = 0;
    long currWindowSum = 0;
    HashSet<Integer> set = new HashSet<>();

    while (j < nums.length) {
        while (set.contains(nums[j])) {
            currWindowSum -= nums[i];
            set.remove(nums[i]);
            i++;
        }

        currWindowSum += nums[j];
        set.add(nums[j]);

        if (j - i + 1 == k) {
            result = Math.max(result, currWindowSum);

            currWindowSum -= nums[i];
            set.remove(nums[i]);
            i++;
        }

        j++;
    }

    return result;
}
```

Intimate

30. Take K of Each Character from left and Right -

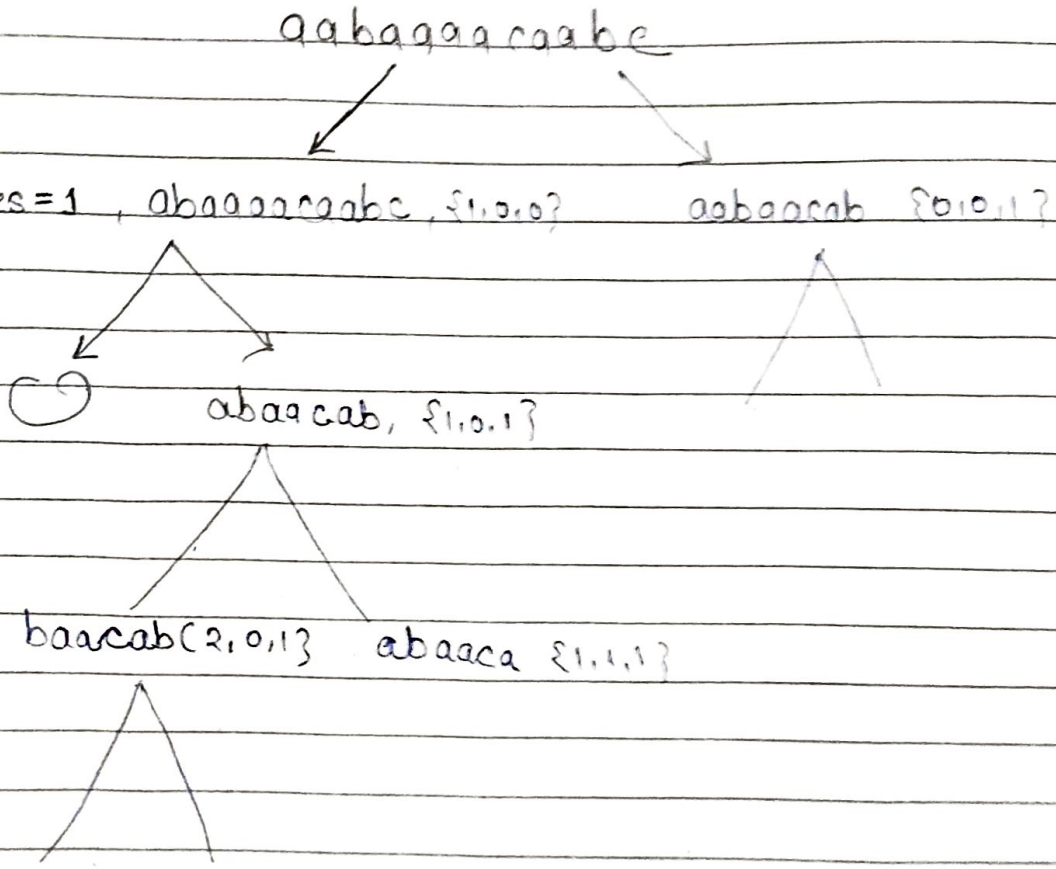
Example:- S = "aabaaaacaabc" K = 2

Output - 8

>= k

a - 4
b = 2
c -> 2

~~Brute force~~



Date / /

Brute force Code:—

```
class Solution {
    private int result = Integer.MAX_VALUE;

    private void solve(String s, int k, int i, int j,
                       int minutes, int[] freq) {
        if (freq[0] >= k && freq[1] >= k &&
                freq[2] >= k) {
            result = Math.min(result, minutes);
            return;
        }
        if (i > j) return;

        int[] tempFreqLeft = freq.clone();
        tempFreqLeft[s.charAt(i) - 'a']++;
        solve(s, k, i+1, j, minutes+1, tempFreqLeft);

        int[] tempFreqRight = freq.clone();
        tempFreqRight[s.charAt(j) - 'a']++;
        solve(s, k, i, j-1, minutes+1, tempFreqRight);
    }

    public int takeCharacters(String s, int k) {
        int[] freq = new int[3];
        int i = 0;
        int j = s.length() - 1;
        int minutes = 0;
        solve(s, k, i, j, minutes, freq);

        return result == Integer.MAX_VALUE ? -1 :
                result;
    }
}
```

Date __/__/__
Intimate

Approach-02
Sliding Window

(1) "g a b g g a a g c a a b c" , K=2
i
j

g = 8 7
b = 2
c = 2

minute = 12

(2) "g g b a a a a c a a b c" K=2
i j

g = 7 6
b = 2
c = 2

minute = 11

(3) "g a b a a a a c a a b c" K=2
i j

g = 6
b = 1
c = 2

minute = 10

(4) "g g b g a a a c a a b c" K=2
i j

g = 7
b = 1
c = 2

minute = 11

(5) "a a b a a a a c a a b c" K=2
i j

g = 7
b = 2
c = 2

minute = 10

Date __/__/__
Intimate

(6) "a a b a a a a c a a b c" , K=2
(i, j)

a = 7
b = 2
c = 2

minutes = 9
windowlength = j - i + 1 ⇒ 1

(7) "a a b a a a a c a a b c" , K=2
(i, j)

a = 6
b = 2
c = 2

minutes = 8
windowlength = j - i + 1 ⇒

(8) "a a b a a a a c a a b c" , K=2
(i, j)

a = 5
b = 2
c = 2

minutes =
windowlength =

(9) "a a b a a a a c a a b c" , K=2
(i, j)

a = 4
b = 2
c = 2

minutes =

(10) "a a b a a a a c a a b c" , K=2
(i, j)

a = 4
b = 2
c = 1 < k { Shrink }

and so on...

Code (Sliding Window) –

```
public int takeCharacters(String s, int k){
    int n = s.length();

    int countA = 0, countB = 0, countC = 0;
    for(char ch : s.toCharArray()){
        if(ch == 'a') countA++;
        else if(ch == 'b') countB++;
        else if(ch == 'c') countC++;
    }

    if(countA < k || countB < k || countC < k){
        return -1;
    }

    int i = 0;
    int j = 0;
    int notDeletedWindowSize = 0;

    while(j < n){
        if(s.charAt(j) == 'a'){
            countA--;
        } else if(s.charAt(j) == 'b'){
            countB--;
        } else if(s.charAt(j) == 'c'){
            countC--;
        }
```

Date __/__/__
Intimate

```
while (i<=j && (countA < k || countB < k ||
                countC < k)) {
    if (s.charAt(i) == 'a') {
        countA++;
    }
    else if (s.charAt(i) == 'b') {
        countB++;
    }
    else if (s.charAt(i) == 'c') {
        countC++;
    }
    i++;
}
notDeletedWindowSize = Math.max(notDeletedWindowSize,
                                j-i+1);
j++;
}
notDeletedWindowSize = Math.max(notDeletedWindowSize,
                                j-i+1);
j++;
}
return n - notDeletedWindowSize;
}
```

Intimate

91. Continuous Subarrays -

Example:-

nums = [5, 4, 2, 4]
0 1 2 3

Output - 8

{5} {4} {2} {4}
{5,4} {4,2} {4,2,4}
{2,4}

0 <= |val| <= 2

How to count Subarrays...

{2, 1, 2}
0 1 2

⇒ 2 - 0 + 1 ⇒ 3

{2} {2,1,2} {2} → 1
{2,1} {1,2} {2,1} → 2
{1} {2} {2,1,2} → 3

How to check if a subarray is continuous -

{2, 3, 2, 4} 0 <= val <= 2

longest = 4
Smallest = 2

|4-2| ⇒ 2 <= 2 ✓

= DRY RUN =

(1) nums = {5, 4, 2, 4}
0 1 2 3
i

max = 5
min = 5

count = 1

}

Date __/__/__
Intimate

(2) nums = {5, 4, 2, 4}
i j 2 3

max = 5
min = 4

5-4 ≠ 1 ✓

count = 1+2 ⇒ 3

{4}, {5, 4}

(3) nums = {5, 4, 2, 4}
i j

Shrink

5-2 ⇒ 3 ✗

(4) nums = {5, 4, 2, 4}
i j

max = 4
min = 2

4-2 ⇒ 2 ✓

count ⇒ 3+2 ⇒ 5

{2} {4, 2}

(5) nums = {5, 4, 2, 4}
i j

max = 4
min = 2

4-4 ⇒ 0

count ⇒ 5+3 ⇒ 8

{4} {4, 4} {4, 2, 4}

(6) nums = {5, 4, 2, 4}
j

Out of Bound

8 Ans.

Date __/__/__
Intimate

// Code (Using TreeMap) -

```
public long continuousSubarrays(int[] nums) {
    int n = nums.length;
    TreeMap<Integer, Integer> map = new TreeMap<>();
    int i = 0, j = 0;
    long count = 0;

    while (j < n) {
        map.put(nums[j], map.getOrDefault(nums[j], 0) + 1);

        while (Math.abs(map.lastKey() -
                        map.firstKey()) > 2) {
            map.put(nums[i], map.get(nums[i]) - 1);

            if (map.get(nums[i]) == 0) {
                map.remove(nums[i]);
            }
            i++;
        }

        count += j - i + 1;
        j++;
    }

    return count;
}
```

Date __/__/__
Intimate

— Approach - 02 —

nums = { 5, 4, 2, 4 }
0 1 2 3

max = maxHeap.top
min = minHeap.top

max-Heap
(4,1)
(4,3)
(2,2)

min-Heap
(2,2)
(4,1)
(4,3)
(5,0)

count = 1 + 2 + 2 + 3 = 8

```
public long continuousSubarrays(int nums[]) {
    int n = nums.length;

    PriorityQueue<int[]> minHeap = new PriorityQueue<>
        ((a,b) -> Integer.compare(a[0], b[0]));

    PriorityQueue<int[]> maxHeap = new PriorityQueue<>
        ((a,b) -> Integer.compare(b[0], a[0]));

    int i = 0, j = 0;
    long count = 0;

    while (j<n) {
        minHeap.offer(new int[] {nums[j], j});
        maxHeap.offer(new int[] {nums[j], j});

        while(Math.abs(maxHeap.peek()[0] -
              minHeap.peek()[0]) > 2) {
            i++;
```

```
while (!maxHeap.isEmpty() &&
        maxHeap.peek() < i) {
    maxHeap.poll();
}

while (!minHeap.isEmpty() &&
        minHeap.peek() < i) {
    minHeap.poll();
}

count += j - i + 1;
j++;
}

return count;

}
```

Date __/__/__
Intimate

// Using Deque

```
public long continuousSubarrays(int nums[]) {
    int l = 0;
    long res = 0;
    Deque<Integer> minD = new LinkedList<>();
    Deque<Integer> maxD = new LinkedList<>();

    for (int r = 0; r < nums.length; r++) {
        while (!minD.isEmpty() &&
               nums[minD.peekLast()] >= nums[r]) {
            minD.pollLast();
        }

        while (!maxD.isEmpty() &&
               nums[maxD.peekLast()] <= nums[r]) {
            maxD.pollLast();
        }

        minD.offerLast(r);
        maxD.offerLast(r);

        while (nums[maxD.peekFirst()] -
               nums[minD.peekFirst()] > 2) {
            l++;
            if (minD.peekFirst() < l) {
                minD.pollFirst();
            }
            if (maxD.peekFirst() < l) {
                maxD.pollFirst();
            }
        }
        res += r - l + 1;
    }
    return res;
}
```

Date __/__/__
Intimate

32. Alternating Groups II – {circular array}

Ex. (1) colors = [0,1,0,1,0], K = 3

Output – 3 {0,1,0} {1,0,1} {0,1,0}

Ex. (2) colors = [0,1,0,0,1,0,1], K = 6

Output – 2 {0,1,0,1,0,1} {1,0,1,0,1,0}

Brute force Dry Run: –

```
public int numberOfAlternatingGroups(int colors[], int k){
    int n = colors.length;
    int[] extended = new int[n+k-1];

    System.
```

Brute force Code:-

```
public int numberOfAlternatingGroups(int[] colors, int k){
    int n = colors.length;
    int count = 0;

    for(int i=0; i<n; i++){
        boolean isAlternating = true;

        for(int j=0; j<k-1; j++){
            int current = colors[(i+j)%n];
            int next = colors[(i+j+1)%n];

            if(current == next){
                isAlternating = false;
                break;
            }
        }

        if(isAlternating){
            count++;
        }
    }

    return count;
}
```

Date / /
Intimate

Using Sliding Window

colors = [0, 0, 1, 0, 1] , K = 3

⇓

[0, 0, 1, 0, 1, 0, 0] , K = 3

{ for circular Handle }

Date / /
Intimate

Code (Sliding Window) —

```
public int numOfAlternatingGroups(int[] colors, int k){
    int n = colors.length;
    int N = n + (k-1);

    int[] extended = new int[N];
    System.arraycopy(colors, 0, extended, 0, n);

    for(int i = 0; i<k-1; i++){
        extended[n+i] = colors[i];
    }

    int result = 0;
    int i = 0, j = 1;

    while(j < N){
        if(extended[j] == extended[j-1]){
            i = j;
            j++;
            continue;
        }
        if(j-i+1 == k){
            result++;
            i++;
        }
    j++;
    }

    return result;
}
```

Date __/__/__
Intimate

— Another Approach —

We need to find the length. So...

(1) [0, 0, 1, 0, 1] , K = 3
 i

length = 1 0 == 0

(2) [0, 0, 1, 0, 1] , K = 3
 i j

0 != 1

length ⇒ 2

(3) [0, 0, 1, 0, 1] , K = 3
 i j

1 != 0

length = 3 == K result = 1

(4) [0, 0, 1, 0, 1] , K = 3
 i j

0 != 1

length = 4 >= K result = 2

(5) [0, 0, 1, 0, 1] , K = 3
 i j

1 != 0

length = 5 >= K result ⇒ 3

(6) [0, 0, 1, 0, 1] , K = 3

0 == 0

length = 1 result ⇒ 3. ✓ Ans.

// Code (2 Pass Solution. Ultra Simple Counting)

```
public int numOfAlternatingGroups(int[] colors, int k) {
    int n = colors.length;
    int result = 0;

    int length = 1;
    int last = colors[0];

    for (int i = 1; i < n; i++) {
        if (colors[i] == last) {
            length = 1;
            last = colors[i];
            continue;
        }
        length++;
        if (length >= k) {
            result++;
        }

        last = colors[i];
    }
    for (int i = 0; i < k-1; i++) {
        if (colors[i] == last) {
            length = 1;
            last = colors[i];
            break;
        }
        length++;
        if (length >= k) result++;

        last = colors[i];
    }

    return result;
}
```

Date __/__/__
Intimate

33. Count of Substrings Containing Every Vowel and K Consonants II –

Ex. Input :- word = "iegouqqieaouqq", k = 1

Output - 3

Explanation – [0...5) = "iegoug"
[6...11) = "qieaou"
[7...12) = "ieaouq"

BRUTE FORCE

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
i	i	e	q	o	u	q	i	i	e	q	o	u	q	q

① All vowels at least once subarray. {T.L.E.}
② consonants == K

Date __/__/__
Intimate

// Code (Sliding Window) -

```
class Solution {
    private boolean isVowel(char ch) {
        return ch == 'a' || ch == 'e' || ch == 'i'
            || ch == 'o' || ch == 'u';
    }

    public long countOfSubstrings(String word,
                                  int k) {
        int n = word.length();
        Map<Character, Integer> mp =
                        new HashMap<>();

        int[] nextCons = new int[n];
        int lastConsIdx = n;

        for (int i = n-1; i >= 0; i--) {
            nextCons[i] = lastConsIdx;

            if (!isVowel(word.charAt(i))) {
                lastConsIdx = i;
            }
        }

        int i = 0;
        int j = 0;
        long count = 0;
        int cons = 0;
```

```
while (j<n){
    char ch = word.charAt(j);
    if ( isVowel (ch) ) {
        mp.put (ch, mp.getOrDefault (ch,0) + 1);
    }else {
        cons++;
    }

    while (cons > k) {
        char ci = word.charAt(i);
        if ( isVowel (ci) ) {
            mp.put (ci, mp.get(ci) - 1);
            if (mp.get(ci) == 0) {
                mp.remove (ci);
            }
        } else {
            cons--;
        }
        i++;
    }

    while (i<n && mp.size() == 5 && cons == k){
        int idx = nextCons[j];
        count += idx - j;

        char ci = word.charAt(i);
        if ( isVowel(ci) ) {
            mp.put (ci, mp.get(ci) -1);
            if ( mp.get(ci) == 0) {
                mp.remove(ci);
            }
        } else {
            cons--;
        }
        i++;  ⇒ अगर
    }
```

Date __/__/__
Intimate

```
        j++;
    }
}
return count;
```

TC -

SC -

Date / / Intimate

34. Number Of Substrings Containing All Three Characters -

Ex. s = "abcabc"

Output - 10

"abc" "bca" "cabc"
"abca" "bcab" "abc"
"abcab" "bcabc"
"abcabc" "cabc"

Brute force

(1) Find All substrings
(2) Checking all of them (a>0, b>0, c>0)

Date __/__/__
Intimate

Dry Run (Sliding Window) —

① "a b c a b c", n = 6

i, j

0	1	2
1	0	0
a	b	c

mp =

result = 0

② a b c a b c

i, j

1	1	0
a	b	c

mp =

result = 0

③ a b c a b c, n = 6

i j

1	1	1
a	b	c

mp =

result ⇒ 4 (6-2)

(n-j) i++

④ a b c a b c

i j

0	1	1
a	b	c

mp =

result ⇒ 4

⑤ a b c a b c

i j

1	1	1
a	b	c

mp =

result = 4 + 3

(6-3) i++

⑥ a b c a b c

i j

1	0	1
a	b	c

mp =

result ⇒ 7

⑦ a b c a b c

i j

1	1	1
a	b	c

mp =

result = 7 + 2 ⇒ 8 (6-4)

and so on ....

Ex:- "aaabc"

```
while(a>0 && b>0 &&
```

Date 6/7/6
Int

```
}
```

// Sliding Window Code:-

```
public int numberOfSubstrings(String s){
    int n = s.length();
    int result = 0;
    int mp[] = new int[3];

    int i=0, j=0;
    while(j<n){

        char ch = s.charAt(j);
        mp[ch-'a']++;

        while(mp[0]>0 && mp[1]>0 &&
              mp[2]>0){

            result += (n-j);
            mp[s.charAt(i)-'a']--;
            i++;
        }

        j++;
    }

    return result;
}
```

Date / / Intimate

35. Longest Nice Subarray –

Ex: nums = [1, 3, 8, 48, 10]

Output = 3

3 & 8 = 0
3 & 48 = 0
8 & 48 = 0

// Brute force :–

```
public boolean isNice(int nums[], int start, int end){
    int mask = 0;
    for(int i = start; i<=end; i++){
        if((mask & nums[i]) != 0){
            return false;
        }
        mask |= nums[i];
    }
    return true;
}

public int longestNiceSubarray(int nums[]){
    int n = nums.length;
    int result = 0;
    for(int i = 0; i<n; i++){
        for(int j = i; j<n; j++){
            if(isNice(nums, i, j)){
                result = Math.max(result, j-i+1);
            }
            else{
                break;
            }
        }
    }
    return result;
}
```

isNice : (i, j, nums)

{1, 3, 8, 48, 10}
0 1 2 3 4

Bit position = 1

3 → 0 0 0 0 1 1
8 → OR 0 0 1 0 0 0
0 0 1 0 1 1

48 → 1 1 0 0 0 0
0 0 0 0 0 0

Better Code:-

```
public int longestNiceSubarray(int[] nums){
    int n = nums.length;
    int result = 0;

    for(int i=0; i<n; i++){
        int mask = 0;
        for(int j=i; j<n; j++){
            if(mask & nums[j] != 0){
                break;
            }
            result = Math.max(result, j-i+1);
            mask = mask | nums[j];
        }
    }
    return result;
}
```

Date / /
Intimate

//Code (Sliding Window) -

```
public int longestNiceSubarray(int[] nums){
    int n = nums.length;
    int i = 0;
    int j = 0;
    int result = 1;
    int mask = 0;

    while(j<n){
        while((mask & nums[j]) != 0){

            mask = (mask ^ nums[i]);
            i++;
        }

        result = Math.max(result, j-i+1);
        mask = (mask | nums[j]);
        j++;
    }

    return result;
}
```

TC -

SC -